Algorytmy blokowe

Agata Wojtowicz

January 2026

1 Zmiany w algorytmie generowania kluczy rundowych

1. Na wejściu zamiast 128-bitowego klucza użytkownika przyjmowane jest 256 bitów klucza:

k0, k1, k2, k3, k4, k5, k6, k7, k8, k9, k10, k11, k12, k13, k14, k15, k16, k17, k18, k19, k20, k21, k22, k23, k24, k25, k26, k27, k28, k29, k30, k31, k32, k33, k34, k35, k36, k37, k38, k39, k40, k41, k42, k43, k44, k45, k46, k47, k48, k49, k50, k51, k52, k53, k54, k55, k56, k57, k58, k59, k60, k61, k62, k63, k64, k65, k66, k67, k68, k69, k70, k71, k72, k73, k74, k75, k76, k77, k78, k79, k80, k81, k82, k83, k84, k85, k86, k87, k88, k89, k90, k91, k92, k93, k94, k95, k96, k97, k98, k99, k100, k101, k102, k103, k104, k105, k106, k107, k108, k109, k110, k111, k112, k113, k114, k115, k116, k117, k118, k119, k120, k121, k122, k123, k124, k125, k126, k127, k128, k129, k130, k131, k132, k133, k134, k135, k136, k137, k138, k139, k140, k141, k142, k143, k144, k145, k146, k147, k148, k149, k150, k151, k152, k153, k154, k155, k156, k157, k158, k159, k160, k161, k162, k163, k164, k165, k166, k167, k168, k169, k170, k171, k172, k173, k174, k175, k176, k177, k178, k179, k180, k181, k182, k183, k184, k185, k186, k187, k188, k189, k190, k191, k192, k193, k194, k195, k196, k197, k198, k199, k200, k201, k202, k203, k204, k205, k206, k207, k208, k209, k210, k211, k212, k213, k214, k215, k216, k217, k218, k219, k220, k221, k222, k223, k224, k225, k226, k227, k228, k229, k230, k231, k232, k233, k234, k235, k236, k237, k238, k239, k240, k241, k242, k243, k244, k245, k246, k247, k248, k249, k250, k251, k252, k253, k254, k255

Do wygenerowania klucza jednej rundy wykorzystywane jest 64 skrzynki Sbox 4x4.

s0, s1, s2, s3, s4, s5, s6, s7, s8, s9, s10, s11, s12, s13, s14, s15, s16, s17, s18, s19, s20, s21, s22, s23, s24, s25, s26, s27, s28, s29, s30, s31, s32, s33, s34, s35, s36, s37, s38, s39, s40, s41, s42, s43, s44, s45, s46, s47, s48, s49, s50, s51, s52, s53, s54, s55, s56, s57, s58, s59, s60, s61, s62, s63, s64, s65, s66, s67, s68, s69, s70, s71, s72, s73, s74, s75, s76, s77, s78, s79, s80, s81, s82, s83, s84, s85, s86, s87, s88, s89, s90, s91, s92, s93, s94, s95, s96, s97, s98, s99, s100, s101, s102, s103, s104, s105, s106, s107, s108, s109, s110, s111, s112, s113, s114, s115, s116, s117, s118, s119, s120, s121, s122, s123, s124, s125, s126, s127, s128, s129, s130, s131, s132, s133, s134, s135, s136, s137, s138, s139, s140, s141, s142, s143, s144, s145, s146, s147, s148, s149, s150, s151, s152, s153, s154, s155, s156, s157, s158, s159, s160, s161, s162, s163, s164, s165, s166, s167, s168, s169, s170, s171, s172, s173, s174, s175, s176, s177, s178, s179, s180, s181, s182, s183, s184, s185, s186, s187, s188, s189, s190, s191, s192, s193, s194, s195, s196, s197, s198, s199, s200, s201, s202, s203, s204, s205, s206, s207, s208, s209, s210, s211, s212, s213, s214, s215, s216, s217, s218, s219, s220,

s221, s222, s223, s224, s225, s226, s227, s228, s229, s230, s231, s232, s233, s234, s235, s236, s237, s238, s239, s240, s241, s242, s243, s244, s245, s246, s247, s248, s249, s250, s251, s252, s253, s254, s255

2. Klucz jest dzielony na części nazywane *keyWords* oznaczane jako k_i. Zamiast na 4 części 32-bitowe, jest dzielony na 8 części 32-bitowych

   $\mathbf{k_1}$: $k_0, \ldots, k_{31}$

   $\mathbf{k_2}$: $k_{32}, \ldots, k_{63}$

   $\mathbf{k_3}$: $k_{64}, \ldots, k_{95}$

   $\mathbf{k_4}$: $k_{96}, \ldots, k_{127}$

   $\mathbf{k_5}$: $k_{128}, \ldots, k_{159}$

   $\mathbf{k_6}$: $k_{160}, \ldots, k_{191}$

   $\mathbf{k_7}$: $k_{192}, \ldots, k_{223}$

   $\mathbf{k_8}$: $k_{224}, \ldots, k_{255}$

3. Generowane jest kolejne 124 składowych 32-bitowych.

Na podstawie wartości utworzonych ***keyWords*** rekurencyjnie generowane są wartości składowe kolejnch kluczy rundowych nazywane *preKeys*. Wzór na kolejne wartości ***preKeys*** to wykonanie operacji xor na wartościach kolejnych części 32-bitowych z wartością φ oraz wartością ineksu generowanego *preKey*:

$$w_i = w_{i-8} \oplus w_{i-7} \oplus w_{i-6} \oplus w_{i-5} \oplus w_{i-4} \oplus w_{i-3} \oplus w_{i-2} \oplus w_{i-1} \oplus \varphi \oplus i \lll 11$$

$$\varphi = \frac{\sqrt{5}+1}{2}$$

Przy czym wartości $w_1, w_2, w_3, w_4, w_5, w_6, w_7, w_8$ odpowiadają wartościom ***keyWords***.

Operacja xor na kolejnych wartościach *keyWords* sprawia, że na wartość w_9 mają wpływ wszystkie bity klucza głównego. Kolejne wartości generowane są rekurencyjnie i każda poprzednia wartość w_{i-1} ma wpływ na następną w_i. To znaczy, że na każdą z wartości *preKeys* mają wpływ wszystkie bity klucza głównego.

4. Przetwarzanie wartości *preKeys* za pomocą Sboxów.

W początkowym algorytmie wykorzystywane było osiem różnych Sboxów to przetwarzania kolejnych wartości. Wykorzystywane Sboxy miały rozmiar $4x4$ - przyjmowały na wejściu 4 wartości *preKeys*. Po wydłużeniu klucza, Sboxy pozostają tego samego rozmiaru, ale zamiast wykorzystywania 8 różnych Sboxów stosowany jest tylko jeden - taki sam dla każdej kolejnej ósemki 256-bitowej. Wykorzystywany Sbox:

$$S = [12, 13, 0, 4, 10, 3, 8, 2, 15, 1, 7, 6, 14, 5, 11, 9]$$

Analogicznie do poprzedniego algorytmu, jedno słowo w_i dzielone jest na 4 bloki po 8 bitów, na takim wejściu wykonywana jest operacja Sbox (przetwarzanie przez Sboxa). W przypadku 128-bitowego klucza

głównego (krótszego) przetwarzane były 4 takie Sboxy równolegle. Dla klucza wydłużonego, potrzebne jest 8 równoległych Sboxów.

$$\{K_7, K_6, K_5, K_4, K_3, K_2, K_1, K_0\} := S(w_0, w_1, w_2, w_3, w_4, w_5, w_6, w_7)$$

$$\{K_{15}, K_{14}, K_{13}, K_{12}, K_{11}, K_{10}, K_9, K_8\} := S_1(w_8, w_9, w_{10}, w_{11}, w_{12}, w_{13}, w_{14}, w_{15})$$

$$\ldots$$

Zmiana w pseudokodzie:

Algorithm 1 Operacja Sbox - poprzednio

```
Function sbox_operation(sbox, w_i):
result ← 0
for j ← 0 to 28 step 4 do
    oneSbox ← (w_i ≫ j) & 0xF
    sbox_output ← sbox[oneSbox]
    result ← result | (sbox_output ≪ j)
end for
return result

sbox_order ← [3, 2, 1, 0, 7, 5, 6, 4]
Function transform_preKeys(keyWords, sbox_order, sboxes):
subKeys ← [ ]
for i ← 0 to 32 do
    sbox_idx ← sbox_order[i (mod 8)]
    sbox ← sboxes[sbox_idx]
    start ← i * 4
    sbox_input ← keyWords[start : start + 4]
    for each w_j in sbox_input do
        subKey ← sbox_operation(sbox, w_j)
        subKeys.append(subKey)
    end for
end for
return subKeys
```

Algorithm 2 Operacja Sbox - aktualna

```
Function sbox_operation(sbox, w_i):
result ← 0
for j ← 0 to 28 step 4 do
    oneSbox ← (w_i ≫ j) & 0xF
    sbox_output ← sbox[oneSbox]
    result ← result | (sbox_output ≪ j)
end for
return result

Function transform_preKeys(keyWords, sbox):
subKeys ← [ ]
for i ← 0 to 32 do
    sbox_input ← keyWords[start : start + 8]
    for each w_j in sbox_input do
        subKey ← sbox_operation(sbox, w_j)
        subKeys.append(subKey)
    end for
end for
return subKeys
```

5. Złączenie składowych w klucze rundowe następuje analogicznie do wersji z krótszym kluczem. Różnica polega na tym, że w przypadku 128-bitowego klucza głównego, klucz rundowy powstawał z 4 wartości (na wyjściu z Sboxa), a w przypadku wydłużonego klucza do 256-bitów, klucz rundowy powstaje z 8 wartości.

2 Analiza przetwarzania klucza głównego

Ponieważ algorytm generowania kluczy rundowych bazuje na Serpencie, w którym klucze rundowe powstają na podstawie rekurencji, w której wartościami początkowymi są słowa złożone ze wszystkich bitów klucza głównego, a każde kolejne wyznaczane są z wykorzystaniem operacji xor, wszystkie bity klucza głównego maja wpływ na klucze rundowe od pierwszej do ostaniej (52) rundy. Oprócz tego każda z wyznaczonych wartości przechodzi przez Sbox. Stąd wniosek, wszystkie klucze rundowe zależą od wszystkich bitów klucza głównego, a każdy z bitów klucza rundowego jest wyjściem z Sboxa.

Klucz pierwszej rundy K_i $(i = 1)$ tworzony jest z wartości k_{8i} do k_{8i+7}, które powstały po przejściu wyrazów (*preKeys*) w_{i8} do w_{i8+7} przez 64 Sboxy.

Klucz pierwszej rundy:

sk0, sk1, sk2, sk3, sk4, sk5, sk6, sk7, sk8, sk9, sk10, sk11, sk12, sk13, sk14, sk15, sk16, sk17, sk18, sk19, sk20, sk21, sk22, sk23, sk24, sk25, sk26, sk27, sk28, sk29, sk30, sk31, sk32, sk33, sk34, sk35, sk36, sk37, sk38, sk39, sk40, sk41, sk42, sk43, sk44, sk45, sk46, sk47, sk48, sk49, sk50, sk51, sk52, sk53, sk54, sk55, sk56, sk57, sk58, sk59, sk60, sk61, sk62, sk63, sk64, sk65, sk66, sk67, sk68, sk69, sk70, sk71, sk72, sk73, sk74, sk75, sk76, sk77, sk78, sk79, sk80, sk81, sk82, sk83, sk84, sk85, sk86, sk87, sk88, sk89, sk90, sk91, sk92, sk93, sk94, sk95, sk96, sk97, sk98, sk99, sk100, sk101, sk102, sk103, sk104, sk105, sk106, sk107, sk108, sk109, sk110, sk111, sk112, sk113, sk114, sk115, sk116, sk117, sk118, sk119, sk120, sk121, sk122, sk123, sk124, sk125, sk126, sk127, sk128, sk129, sk130, sk131, sk132, sk133, sk134, sk135, sk136, sk137, sk138, sk139, sk140, sk141, sk142, sk143, sk144, sk145, sk146, sk147, sk148, sk149, sk150, sk151, sk152, sk153, sk154, sk155, sk156, sk157, sk158, sk159, sk160, sk161, sk162, sk163, sk164, sk165, sk166, sk167, sk168, sk169, sk170, sk171, sk172, sk173, sk174, sk175, sk176, sk177, sk178, sk179, sk180, sk181, sk182, sk183, sk184, sk185, sk186, sk187, sk188, sk189, sk190, sk191, sk192, sk193, sk194, sk195, sk196, sk197, sk198, sk199, sk200, sk201, sk202, sk203, sk204, sk205, sk206, sk207, sk208, sk209, sk210, sk211, sk212, sk213, sk214, sk215, sk216, sk217, sk218, sk219, sk220, sk221, sk222, sk223, sk224, sk225, sk226, sk227, sk228, sk229, sk230, sk231, sk232, sk233, sk234, sk235, sk236, sk237, sk238, sk239, sk240, sk241, sk242, sk243, sk244, sk245, sk246, sk247, sk248, sk249, sk250, sk251, sk252, sk253, sk254, sk255

Każdy kolejny klucz rundy będzie reprezentowany takim samym wzorem (w tym ostatni), ponieważ każdy kolejny wyraz k_i z któego jest tworzony zależy od poprzedniego. Aby przetworzyć jeden *preKey* (w_i) wykorzystywane są 4 Sboxy:

$w_i =$ sk0, sk1, sk2, sk3, sk4, sk5, sk6, sk7, sk8, sk9, sk10, sk11, sk12, sk13, sk14, sk15, sk16, sk17, sk18, sk19, sk20, sk21, sk22, sk23, sk24, sk25, sk26, sk27, sk28, sk29, sk30, sk31

Stąd po przejściu w_{i8} do w_{i8+7} przez 8 równoległych Sboxów - wszystkie bity klucza rundowego przeszły przez funkcję nieliniową.

3 Wyznaczenie liczby rund zgodnie z nowymi parametrami bezpieczeństwa

maksymalna korelacja wejściowo-wyjściowa aktywnego SBoxa	$\boldsymbol{\alpha = 2^{-1}}$
maksymalne prawdopodobieństwo, że niezerowa różnica wejściowa daje ustaloną różnicę wyjściową	$\boldsymbol{\delta = 2^{-2}}$
dolne ograniczenie na liczbę aktywnych Sboxów	$\boldsymbol{B = 5}$
długość bloku wejściowego	$\boldsymbol{128b}$
długość wydłużonego klucza	$\boldsymbol{2k = 256b}$
złożoność ataku	$\boldsymbol{2^{256}}$

prawdopodobieństwo najlepszej charakterystyki różnicowej	$\boldsymbol{p = \delta^B = 2^{-10}}$
największa korelacja wejściowo-wyjściowa	$\boldsymbol{c = \alpha^B = 2^{-5}}$

liczba wymaganych do ataku różnicowego wybranych tekstów jawnych dla dwóch rund	$\boldsymbol{p^{-1} = 2^{10}}$
liczba wymaganych do ataku liniowego wybranych tekstów jawnych dla dwóch rund	$\boldsymbol{c^{-2} = 2^{10}}$

W celu zabezpieczenia szyfru przed kryptoanalizą liniową i różnicową, minimalna liczba rund wynosi **52** rundy, zamiast 26 (jak w przypadku krótszego klucza głównego)

- $(p^{-1})^{26} = 2^{260} > 2^{256}$
- $(c^{-2})^{26} = 2^{260} > 2^{256}$